

Verifiable Containment Attestation After LLM Guardrail Failure

Anonymous Author(s)

ABSTRACT

LLM guardrails are commonly evaluated at the point where a user input is accepted or blocked. Agentic failures, however, can occur after this first gate has opened: untrusted retrieved content may gain authority, model output may request a tool, or unsafe text may be exported. This paper presents a provider-agnostic post-guardrail containment and evidence system. The prototype enforces input, context-provenance, tool-invocation, and output-leakage boundaries, then emits metadata-only evidence bundles signed with Ed25519 over canonical JSON. Our motivating observation uses a real input-only guardrail: a frozen live Llama Guard 4 12B capture allowed 138 of 150 malicious cases on a 180-case corpus, not because it is weak but because the malicious instruction is structurally located downstream of the user turn it inspects. The gateway then contained all 138 downstream (contained fraction 138/138, 95% Clopper-Pearson CI [0.97, 1.0]). On the same corpus the gateway blocked 30/30 direct-input attacks, passed 30 adversarially-shaped benign hard negatives with no false blocks (0/30, 95% CI [0, 0.12]), and recorded no unauthorized tool execution, context-authority escalation, or unsafe export. On an independently-authored attack corpus rendered from the AgentDojo benchmark (175 payloads we did not write), the deterministic input firewall detects only 35/175 (95% CI [0.14, 0.27]) yet all 175 are structurally contained downstream with no untrusted context gaining authority—the same thesis under third-party attacks. A signed VCA bundle verifies offline and rejects tampering, and a synthetic recorded-signal fixture confirms that downstream containment is invariant to the external advisory verdict. On a live, fooled Llama-3.3-70B agent (AgentDojo, 140 attacked cases, action classes pre-registered before results), advisory provenance demotion does not contain, but structural authority gating contains all baseline successful unauthorised egress and destructive-mutation side effects (targeted attack success 9/140 to 0/140) within the declared taxonomy. We are deliberate about scope: the corpus is synthetic and self-authored, the single real external guardrail is one model, and the signature proves issuer and integrity, not producer honesty, jailbreak immunity, production readiness, model safety, or vendor ranking.

CCS CONCEPTS

• Security and privacy → Systems security; • Computing methodologies → Artificial intelligence.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

AISeC '26, October 2026, TBD

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

KEYWORDS

LLM security, prompt injection, agentic systems, guardrails, attestation, reproducibility

ACM Reference Format:

Anonymous Author(s). 2026. Verifiable Containment Attestation After LLM Guardrail Failure. In *19th ACM Workshop on Artificial Intelligence and Security (AISeC '26)*. ACM, New York, NY, USA, 8 pages.

1 INTRODUCTION

Input guardrails are necessary, but they are not a complete security boundary for agentic LLM systems. A direct jailbreak may be visible in the user input. An indirect prompt injection may instead arrive through retrieved data, tool output, or model-generated text that is later interpreted as an action request. Prior work shows that LLM-integrated applications can treat untrusted data as instructions [3]; benchmark environments such as AgentDojo show that tool-using agents require explicit security evaluation under untrusted context [2].

This paper studies a narrower systems question: after a guardrail misses, can a gateway produce machine-checkable evidence of what did and did not happen downstream? We do not propose a better jailbreak detector. We ask whether downstream containment can be made explicit, replayable, and cryptographically verifiable. The intended reader is a reviewer, operator, or auditor who should not have to trust a live service to know whether untrusted context gained authority, an unauthorized tool executed, or unsafe output was exported.

The VCA prototype enforces four boundaries: an input firewall, a context-provenance guard, a tool-invocation gate, and an output-leakage firewall. It treats model output and external guardrail verdicts as advisory. Each run emits metadata-only receipts and audit events. A second layer packages run-set evidence as canonical JSON and signs it with Ed25519 [4, 7], using public verification keys and deterministic reproduction checks.

We make three contributions:

- (1) A post-guardrail containment gateway that makes downstream consequences—context-authority escalation, tool self-authorization, and unsafe export—the unit of evaluation, with boundary decisions taken by deterministic code rather than the model under inspection.
- (2) A verifiable containment attestation format that turns those boundary outcomes into metadata-only canonical evidence bundles signed with Ed25519, with public verification, offline reproduction, tamper rejection, and machine-readable non-claims, under an explicitly stated honest-producer trust boundary.
- (3) An evidence-first evaluation, including a real input-only guardrail capture that quantifies the downstream blind spot, a synthetic advisory-invariance check, and a public reproduction chain, paired with a claim-governance pattern that prevents bounded evidence from being read as a broad safety claim.

2 BACKGROUND AND MOTIVATION

Prompt injection is a first-order LLM application risk in practitioner guidance such as OWASP’s LLM Top 10 [6]. The key distinction for this work is not whether prompt injection exists, but where it enters an agentic workflow. Direct injection targets the user turn. Indirect injection targets content later retrieved or processed by the application. Tool-using agents add a further boundary because model output can be interpreted as an action request unless a separate policy gate checks authorization.

Most guardrail evaluations report whether an input is blocked, warned, or allowed. That is useful, but it does not answer the downstream consequence question. If an input-only guardrail allows a benign-looking task while the malicious instruction lives in untrusted context, then the relevant security property is whether that context can gain authority. If a model asks to execute a tool, the relevant property is whether the tool request is authorized outside the model. If model output contains unsafe material, the relevant property is whether it reaches the export boundary.

The VCA prototype therefore treats containment evidence as the primary artifact. It records boundary outcomes without publishing raw harmful transcripts or raw model outputs. This evidence-first framing is also a reproducibility requirement: a reviewer should be able to verify the artifact offline, with public keys and scripts, rather than trusting a live provider or private deployment.

3 RELATED WORK

Prompt injection in LLM applications. Indirect prompt injection showed that LLM-integrated applications blur the boundary between data and instructions: content retrieved from email, webpages, documents, or plugins can become operational instruction text once it reaches the model [3]. USENIX Security work has since formalized prompt-injection attacks and defenses, showing that case studies alone are insufficient and that defenses should be evaluated against explicit tasks, attack classes, and security properties [5]. Practitioner taxonomies, including OWASP LLM01, similarly separate direct and indirect injection and emphasize least privilege, output validation, human approval for high-risk actions, and segregation of untrusted content [6]. Our work follows this systems view but evaluates a different question: after the input guardrail misses, can downstream containment and its evidence be verified independently?

Agent benchmarks and tool-use security. AgentDojo evaluates LLM agents that operate tools over untrusted data and provides an extensible benchmark with realistic tasks, attacks, and defenses [2]. This line of work is essential for measuring whether agents complete tasks while resisting prompt injection. The VCA prototype is complementary: it does not claim a broad benchmark or a better attack detector. Instead, it records whether specific downstream consequences occurred, including context authority escalation, unauthorized tool execution, and unsafe output export, and then signs the evidence for offline verification.

Capability-based agent defences. A complementary line argues that prompt injection should be defeated by construction rather than detection: CaMeL extracts the control and data flow from the trusted user query and uses capabilities so that untrusted data cannot alter program flow, reporting provable security on a

subset of AgentDojo [1]. Our live-agent results are in this lineage—a fooled model proposes actions and a structural authority gate decides them—but Simurgh targets a different artefact. We do not claim capability gating as novel; we contribute pre-registered, by-class, metadata-only *evidence* of each bounded authorisation decision, with explicit non-claims about what was not proved.

Guardrails and audit evidence. Guardrails are usually evaluated as online decisions: input accepted, warned, or blocked. That frame is useful but incomplete for agentic systems because security-relevant outcomes can happen after acceptance. Our design borrows from security audit practice: preserve metadata, bind evidence with hashes and signatures, verify offline, and state non-claims. The use of canonical JSON and Ed25519 follows established standards for deterministic signing and verification [4, 7]. The novelty is not the cryptography itself; it is applying signed, machine-readable containment evidence to post-guardrail LLM security claims.

4 THREAT MODEL AND NON-CLAIMS

The defender operates an agentic LLM gateway between user input, retrieved context, model output, tool execution, and final export. The assets are tool authority, confidential context, system/developer instructions, unsafe output surfaces, and the integrity of exported evidence. The adversary can submit direct prompt-injection text, place malicious instructions in untrusted context, cause unauthorized tool requests, attempt output leakage, and tamper with exported evidence after publication.

Out of scope are kernel compromise, malicious system operators, private-key theft, production deployment assurance, and formal proof of semantic safety. Signed evidence proves that an exported bundle was issued by the signing-key holder and was not modified after signing; it does not prove that the live service was uncompromised, that the signing key was never stolen, or that all possible attacks were covered. Table 1 records the main paper claims and their boundaries.

5 SYSTEM DESIGN

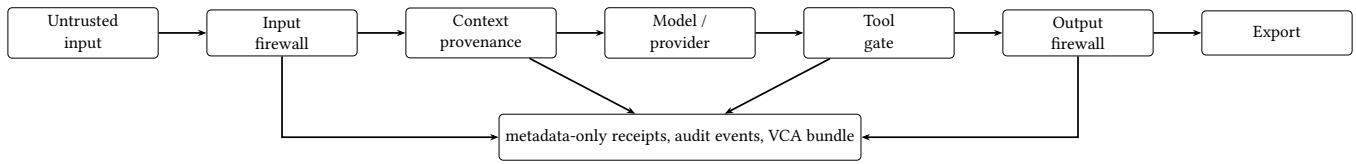
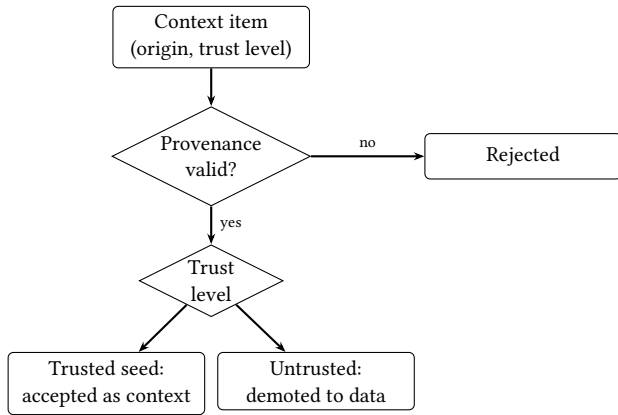
Figure 1 shows the gateway. The input firewall handles direct attacks. If input passes, the context-provenance guard classifies each context item by origin and trust level; untrusted context can be passed as data but cannot acquire system or developer authority. Provider output is treated as untrusted material. Tool requests must pass the tool gate; final output must pass the output firewall before export. Table 2 summarizes each boundary, the attack class it addresses, its enforcement action, and the evidence it emits.

The design intentionally avoids using the model as its own security monitor. Boundary decisions are made by deterministic gateway code over structured metadata. The model and any external guardrail can influence the ordinary application response, but neither can authorize a tool, relabel untrusted context as trusted instruction, or suppress an audit event. This separation is what makes the later attestation meaningful: the evidence records gateway decisions rather than model assertions about those decisions.

Figure 2 shows the context-provenance authority decision in more detail. A context item carries an origin and a declared trust level; an invalid or unrecognized provenance is rejected, a trusted seed is accepted as context, and any untrusted origin (for example a

Table 1: Claims and non-claims. Evidence paths are repository-local and are verified in the paper package audit.

Claim	Evidence source	Non-claim boundary
Input-miss cases were downstream-contained on the frozen 180-case corpus	stage-3l/metrics.json	Does not imply general jailbreak resistance or production traffic validation.
The Stage 3M VCA bundle verifies offline and rejects tampering	stage-3m/verifier-output.txt and attestation files	Does not make signed evidence factual truth or prove the service was uncompromised.
The real external guardrail (Llama Guard 4) is one input-only reference; Stage 3V-A is a synthetic advisory-invariance fixture	stage-3v-b/metrics.json, stage-3v/metrics.json	One model, self-reported capture; the fixture carries no detection claim; not a vendor ranking.
The public VCA chain is replayable at tiered depths	stage-3x/vca-chain-reproduction-res	Not a full 12/12 full reproduction.
Structural authority gating contained all baseline successful attacks on a fooled live agent within the pre-registered taxonomy	stage-1-live/llama-3.3-70b-fp8/author	Not a jailbreak immunity or injection prevention; taxonomy excludes non-destructive mutation, financial, and code actions; one benign regression.

**Figure 1: Post-guardrail containment pipeline. The defense acts after an input guardrail can fail. Provider output and external guardrail signals are advisory; final claims are derived from boundary evidence.****Figure 2: Context-provenance authority decision. Untrusted context may flow as data but never acquires system or developer authority, and neither provider output nor an external guardrail verdict can grant authority or relabel context. This deterministic decision—not the model—produces the boundary outcome that is later attested.**

tool result, an external reference, or provider output) is demoted to data so that it can inform the response but cannot acquire system or developer authority. Because this decision is made over structured metadata rather than free text, its outcome is recorded verbatim in the receipt, which is what a later verifier checks.

Evidence schema. Each boundary emits a compact record with a run identifier, boundary name, policy version, input/output digests where applicable, verdict, reason code, and non-claim tags. Raw

prompts and raw provider outputs are excluded from the evidence packet. The VCA bundle aggregates these records with corpus metadata, metric summaries, privacy reports, referenced evidence hashes, public verification metadata, and a machine-readable list of limitations. This lets a reviewer check both positive claims, such as “no unauthorized tool execution in this corpus,” and negative scope boundaries, such as “not a production deployment claim.”

6 VERIFIABLE CONTAINMENT ATTESTATION

The attestation layer packages a run set as metadata-only evidence. A bundle contains metrics, boundary breakdowns, policy digests, privacy reports, referenced evidence hashes, and machine-readable non-claims. The signature covers canonical JSON, allowing a verifier to check the signed content independently of whitespace or formatting changes. Verification has a portable mode for schema, signature, digest, public-key fingerprint, referenced file hashes, gate results, and leakage scans. Reproduction mode reruns deterministic producers where available.

7 EVALUATION

We use only repository evidence that can be traced to files and, where practical, commands. The evaluation ladder is shown in Figure 3; Table 3 summarizes the main results and Table 6 the reproducibility and tamper checks.

Stage 3L. A deterministic 180-case corpus is driven through real gateway boundary functions in pipeline order: prompt classification, context-provenance checks, tool gating, and output scanning. The corpus contains 120 input-miss downstream cases, 30 direct-input attacks, and 30 benign hard negatives. The main dependent variables are consequence metrics: whether untrusted context gained authority, whether an unauthorized tool executed, whether unsafe

Table 2: Containment boundaries.

Boundary	Attack class	Enforcement action	Evidence emitted
Input firewall	Direct prompt injection	Block or warn before provider call	Input verdict, receipt, audit event
Context provenance	Indirect prompt injection	Demote untrusted context to data	Context verdict and boundary label
Tool gate	Tool self-authorization	Refuse unauthorized request	Tool verdict and no-execution evidence
Output firewall	Leakage/export pressure	Block unsafe export	Output verdict and output hash metadata

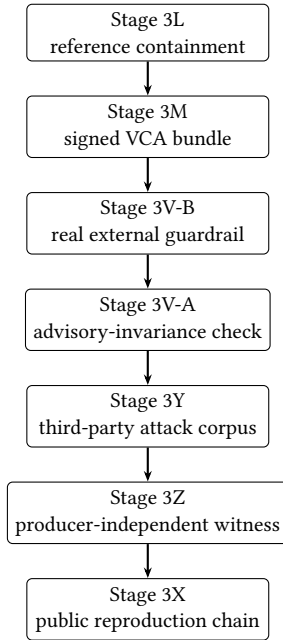


Figure 3: Evaluation ladder. Each stage contributes a different part of the paper claim and has a separate evidence artifact.

output was exported, whether receipts were emitted, and whether the audit chain verified. Because the corpus is synthetic and authored by us, the perfect malicious counts (120/120 input-miss contained, 30/30 direct blocked, 0/150 targeted attack success) are fixture-validity checks—they confirm the boundaries fire exactly as the case design intends—and not estimates of any real-world attack rate. To check that containment is not achieved by indiscriminate blocking, we report a benign false-positive rate over the 30 hard negatives, all adversarially shaped to resemble the malicious families while remaining legitimate; none were blocked (0/30). We attach exact 95% Clopper-Pearson intervals throughout because the per-cell sample sizes are small: 0/30 bounds the benign false-positive rate at $[0, 0.12]$ and 0/150 bounds targeted success at $[0, 0.02]$, so these are upper-bounded near-zero results, not demonstrated true zeros.

Containment of the 120 input-miss cases is distributed across the downstream boundaries rather than concentrated at a single gate (Table 5): the context-provenance guard contained 72 cases, the tool gate 24, and the output firewall 24. The input firewall contained none of these by construction, because each input-miss case is

designed to pass the input gate so that the downstream boundary under test is the one exercised.

Stage 3M. The Stage 3L run set is packaged into a metadata-only bundle signed with Ed25519. The public key and expected fingerprint are committed; the private key is not. The verifier checks structure, signature, evidence-file hashes, gate results, privacy, and reproduction matches. Tamper cases modify signed content, referenced metrics, and gate summaries to confirm that verification fails closed.

Stage 3V-B (the real external guardrail). A single real input-only guardrail anchors the external observation: a frozen live Llama Guard 4 12B capture (meta-llama/Llama-Guard-4-12B, one input-only greedy pass, temperature 0, 20 new tokens, 8-bit quantization, one NVIDIA RTX 3090 24GB GPU, transformers v4.51.3 Llama-Guard preview, PyTorch 2.8.0, CUDA 12.8). As an input-only signal it allowed 138 of 150 malicious cases. We read this not as a weakness of the model but as the structural blind spot of any input-only guardrail: in the 120 input-miss cases the malicious instruction is located downstream of the user turn the guardrail inspects. The gateway then contained all 138 downstream (138/138, 95% CI $[0.97, 1.0]$), and external-plus-gateway targeted success was 0/150. The frozen capture, not the model, is replayed in CI (model_reexecuted_in_ci is false); the live capture origin is self-reported and signed into evidence but not independently re-derived. This is one model and a boundary study, not a vendor comparison or ranking.

Stage 3V-A (advisory-invariance check, not a second guardrail). We have only one real external guardrail, so we do not claim a second comparator. Instead we use a synthetic, deterministic recorded-signal fixture—spanning allow, warn, and block verdicts that we authored—to test a structural property of the design: the gateway’s downstream containment outcome does not depend on the external advisory verdict. Across the fixture the containment outcome was identical regardless of whether the synthetic signal said allow, warn, or block. This advisory-invariance is the property that lets the gateway treat any external guardrail, including a future stronger one, as untrusted advice rather than as authority. The fixture’s verdicts are authored by us and carry no detection claim, so we report no attack-success rate for it.

Stage 3Y (independently-authored attack corpus). To break the self-authored circularity of Stage 3L, we render a third-party attack corpus from the AgentDojo benchmark [2]: 35 injection-task goals across four suites combined with five published attack envelopes (ignore_previous, system_message, injeacagent, important_instruction_direct), for 175 payloads authored by the AgentDojo team, not by us. We drive them through the real boundaries and separate content detection from structural containment (Table 4). The deterministic input firewall detects only 35/175 (95% CI $[0.14, 0.27]$)—just the

injecagent family, which contains an explicit override phrase—and misses 140/175 [0.73, 0.86]; even `ignore_previous` evades it because of a word insertion and a typo in AgentDojo’s own string. The same 175 payloads, arriving as untrusted context, are structurally contained 175/175 [0.98, 1.0] with zero cases of untrusted context gaining authority, because provenance demotion is content-agnostic. This result is deliberately unflattering to the input layer and is the paper’s thesis under independently-authored attacks: input guardrails miss, and downstream structural containment is what holds. Evidence is metadata-only; raw payloads are not committed.

Stage 3Z (producer-independent witness). This stage operationalizes the honest-producer boundary from the security analysis. An independent witness cross-checks each signed receipt against a consequence oracle (canary sightings at the real sinks) that is not derived from the receipt. Over a four-fixture self-proof—two honest runs and two dishonest runs that sign clean receipts for a real canary leak and a hidden tool execution—the witness corroborated 2/2 honest runs and flagged 2/2 dishonest runs, with 0 false accusations and 0 missed lies. Crucially, the dishonest receipt’s Ed25519 signature still verifies, so the plain verifier passes while the witness raises the conflict: the falsification holds.

Stage 3X. A public VCA timeline pins 12 signed rungs by tag and commit. We are explicit that “12/12” means twelve rungs verified at their declared tier, not twelve full re-executions: 3 rungs are re-run end-to-end, 7 are checked by evidence-hash, and 2 are index-only. The guarantee that holds across all three tiers, and across the historical evidence-hash formats, is the format-independent evidence-root digest.

Stage 1-LIVE (live-agent authority gating). The corpora above are deterministic or self-authored. To test whether containment holds when a real model is actually fooled, we run the pinned AgentDojo workspace suite with a live agent: Llama-3.3-70B-Instruct (FP8) under greedy decoding, driven by the canonical `important_instructions` attack across all 14 injection goals $\times$ 10 user tasks (140 attacked cases). The injection action classes are pre-registered from each task’s ground truth before any defended run (egress, egress+delete, mass-recipient egress, and a single delete-only goal). We compare three in-loop defences against the same baseline (Table 7). *Demotion*, which wraps untrusted tool output and instructs the model to treat it as data, is advisory and does not contain: targeted attack success falls only 10/140 $\rightarrow$ 8/140 while benign utility drops 8/10 $\rightarrow$ 6/10. A structural *egress gate*, which blocks send/share/invite actions whose destination is not grounded in the trusted task, contains every egress-class success (9/140 $\rightarrow$ 4/140) with no benign regression; the four survivors are exactly the delete-only goal, outside its jurisdiction. Adding a *destructive-mutation gate* (the *authority gate*) closes that gap: targeted success reaches 0/140—every baseline successful attack contained—at a cost of one benign regression that is most likely run-to-run nondeterminism (its recipients are named in the task). The headline is precise and falsifiable: *within the pre-registered egress and destructive-mutation taxonomy, structural authority gating contained all baseline successful unauthorised side effects on a fooled live agent while preserving near-baseline benign utility*. This is not jailbreak immunity, not injection prevention, and the taxonomy excludes non-destructive mutation, financial, and code-execution actions.

Stage 4A-lite (capability kernel and signed authorisation evidence). The two gates above are bespoke. We refactor them into a single *minimal capability kernel*: a proposed tool call is mapped to typed actions, and one task-grounded capability authorises each (allow/block), so egress and destructive mutation become policy instances rather than separate control flow. The refactor is constrained to change no behaviour: an exhaustive differential test proves the kernel-backed gate is byte-identical to the preserved pre-refactor gate across every branch, argument shape, and trusted-text class, so the Stage 1-LIVE result transfers *only through this equivalence*—we do not re-run the live model, and we do not reconstruct live per-action traces (the frozen run was outcome-recorded, not per-action-recorded). The kernel’s authorisation decisions are then sealed as offline-verifiable evidence: run over a model-free action corpus derived from the pre-registered injection taxonomy, the kernel emits one metadata-only authorisation record per action (typed action, hashed target, capability checked, verdict), bound into an Ed25519-signed bundle with a two-tier verifier that recomputes the decision digest and rejects tampered metrics, edited verdicts, a wrong key, or a stripped non-claim. This is deliberately an evidence-architecture step, not a production-gateway claim: no live re-run, no new action families, and target hashes are metadata-minimisation identifiers, not secrecy against dictionary reconstruction of known public targets.

Stage 4B–4C (laundering-resistant recovery and provenance gating). Two additive policy rungs build on the frozen kernel without changing it. Stage 4B grounds egress not only in the task text but in typed trusted intent sources (the current meeting participants and a selected contact group), recovering the one documented benign over-block; measured against per-target rather than per-case malice, a legitimate participant inside an otherwise-malicious call is correctly allowed while the attacker recipient is blocked, with zero laundering. Stage 4C closes the trust-propagation edge this opens: once intent sources are populated from tool output, an injection can launder the attacker address in as a participant, so each intent entry is gated by provenance and only trusted origins grant authority. A provenance-blind implementation would admit the poisoned-participant attack; provenance gating contains every instance with no laundering and no loss of the legitimate recovery. Each rung is sealed as an independently signed bundle whose manifest binds the previous rung by hash, forming a chain from Stage 4C to 4B to 4A to the frozen Stage 1-LIVE evidence. In a fresh clone of the pushed branch, the Stage-4 evidence chain regenerated all committed artifacts byte-identically, verified all three signed bundles using only public keys, and rejected tampered headline results—preserving the live result through digest binding and differential equivalence rather than live-model replay. We do not claim third-party audit, formal verification, or production readiness; 4C’s provenance labels are modelled rather than derived from a live trust tracker, which we leave to future work.

8 SECURITY ANALYSIS

The system prevents a bounded class of downstream consequences in the evaluated corpora: untrusted context gaining authority, unauthorized tool execution, and unsafe output export. It does so by

Table 3: Main evaluation results.

Evaluation	Corpus / evidence	Key metric	Result	Limitation
Stage 3L	Frozen 180-case reference corpus	Input-miss downstream containment	120/120 (CI [0.97,1])	Synthetic, self-authored corpus; fixture-validity.
Stage 3L	Same	Direct-input attacks blocked	30/30	Does not imply general detector robustness.
Stage 3L	Same	Benign false-positive rate	0/30 (CI [0,0.12])	Hard-negative control; small- n upper bound.
Stage 3L	Same	Targeted attack success	0/150 (CI [0,0.02])	Self-authored corpus; not a world rate.
Stage 3L	Same	Unsafe tool/export/context escalation	0/0/0	Consequence metrics, not semantic safety proof.
Stage 3M	Signed VCA bundle	Offline verification	PASS	Signs evidence under honest-producer assumption; not factual truth.
Stage 3V-B	Real input-only guardrail (Llama Guard 4 12B)	Downstream-contained after guardrail miss	138/138 (CI [0.97,1])	One model; capture self-reported; not a ranking.
Stage 3V-A	Synthetic recorded-signal fixture	Advisory-invariance	invariant	Authored verdicts; no detection claim.
Stage 3Y	Third-party AgentDojo corpus (175)	Input detection / structural containment	35/175 / 175/175	Component-level; not a live agent.
Stage 3Z	Producer-independent witness	Dishonest receipts caught (signature still valid)	2/2	Needs an independent oracle; mechanism + demo.
Stage 3X	Public reproduction packet	Rungs passed	12/12	Tiered replay (3 reproduce / 7 hash / 2 index).

Table 4: Stage 3Y: independently-authored AgentDojo attack corpus (175 payloads) through the real boundaries. Content detection is by the deterministic input firewall; structural containment is content-agnostic provenance demotion. Intervals are 95% Clopper-Pearson.

Boundary	Metric	Result (95% CI)	Reading
Input firewall	Content detection	35/175 [0.14, 0.27]	Only injecagent (explicit override phrase).
Input firewall	Missed	140/175 [0.73, 0.86]	Four of five envelopes evade the denylist.
Context provenance	Structural containment	175/175 [0.98, 1.0]	Untrusted demoted (170) or rejected (5).
Context provenance	Untrusted gained authority	0	Core invariant holds on third-party attacks.

separating model text from policy authority. The evidence layer makes the resulting containment claim checkable offline.

The honest-producer trust boundary. The sharpest limit of any attestation is that it binds integrity and issuer, not truth. The gateway and its evidence producer sit inside the trusted computing base: a signature proves that the bundle was issued by the key holder and not modified afterward, but it cannot prove that the gateway emitted a faithful receipt for the run it actually performed. A gateway that executed an unsafe action and then signed a clean

receipt would pass every check in this paper. We state this assumption explicitly rather than hide it: VCA is meaningful only insofar as the producer is honest and the signing key is uncompromised. Two design choices narrow, but do not close, this gap. First, boundary outcomes are computed by deterministic code over structured metadata, so the receipt records a mechanical decision rather than a model self-report. Second, the bundle binds referenced evidence by hash and the verifier recomputes those hashes, so post-hoc edits are rejected. To go further, Stage 3Z adds a producer-independent

Table 5: Per-boundary containment of the 120 Stage 3L input-miss cases. The input firewall contains none by construction; each case is built to pass the input gate and exercise one downstream boundary.

Downstream boundary	Input-miss cases contained
Context-provenance guard	72
Tool gate	24
Output firewall	24
Total downstream-contained	120

Table 6: Reproducibility and tamper checks.

Check	Expected	Verified	Command / source
Stage 3M verifier	PASS	PASS	stage-3m/verifier-output.txt
Evidence leakage	zero	true	Stage 3M verifier output
Bundle digest	match	true	Stage 3M verifier output
Stage 3X timeline	PASS	true	vca-chain-reproduction-results.json
Tier summary	3 reproduce, 7 hash, 2 index	matched	Stage 3X results JSON

Table 7: Stage 1-LIVE: live Llama-3.3-70B agent on AgentDojo workspace (140 attacked cases, classes pre-registered before results). Demotion is advisory and fails; structural authority gating contains all baseline successes within the declared taxonomy. Same-session baseline; greedy decoding is not bit-deterministic under batched serving, hence the ± 1 baseline drift.

Defence	Mechanism	Targeted ASR	Benign utility
Demotion	Advisory: wrap untrusted tool output as data	10/140 $\rightarrow$ 8/140	8/10 $\rightarrow$ 6/10
Egress gate	Structural: block egress to a destination not grounded in the trusted task	9/140 $\rightarrow$ 4/140	7/10 $\rightarrow$ 7/10
Authority gate	Egress + destructive-mutation gate	9/140 $\rightarrow$ 0/140	7/10 $\rightarrow$ 6/10

witness: it cross-checks the signed receipt against a consequence oracle whose observation channel is not derived from the receipt (canary/honeytoken sightings at the real export and tool sinks). In a falsifiable self-proof, a dishonest gateway signs a clean receipt for a run that actually leaks a canary; its Ed25519 signature still verifies—so the plain verifier is fooled—yet the witness raises a claim conflict, with zero false accusations on honest runs and zero missed lies across the fixtures. The witness assumes a genuinely independent oracle; if the oracle is also compromised the gap reopens, so we present a mechanism and demonstration, not a deployed egress monitor.

The system does not prove semantic harmlessness. An attacker outside the threat model could compromise the signing key, the host, or the evidence producer. A malicious operator could choose not to record evidence. A broader real-world deployment would also need operational key management, monitoring, and incident response. These are deployment concerns outside the paper’s research prototype claim.

9 ETHICS AND RESPONSIBLE RELEASE

This is defensive research. The paper and artifact avoid releasing raw harmful jailbreak transcripts or operational payloads. The corpora are synthetic or redacted. The evidence is metadata-only and omits raw prompts, raw provider outputs, private keys, and real user data. The external guardrail study is vendor-neutral and must not be read as a ranking. The prototype is not deployment-ready

or compliance-approved. These boundaries are stated in the paper, source notes, and artifact README.

10 LIMITATIONS AND FUTURE WORK

The main external-validity limitation is that the evaluation uses frozen synthetic corpora and replay artifacts, not production traffic; the perfect per-cell counts are fixture-validity checks on a self-authored corpus, reported with 95% Clopper-Pearson intervals that remain upper-bounded near-zero results on small samples rather than demonstrated true zeros. The external observation rests on a single real guardrail (one model); the second external artifact is a synthetic advisory-invariance fixture, not an independent guardrail, so we make no multi-guardrail or comparative claim. The live model capture origin is self-reported and signed into evidence, but the verifier does not rehash model weights or prove the original GPU environment. Stage 3X reports uneven replay depth; this is honest, but it means the public chain is not uniformly fully reproduced. As the security analysis states, the attestation assumes an honest evidence producer: a compromised or dishonest gateway is outside what a signature alone can detect. Stage 3Z demonstrates a producer-independent witness that catches a dishonest clean receipt, but it assumes the consequence oracle is itself independent and uncompromised; deploying a real egress/tool oracle is future work. So is third-party artifact evaluation, broader and non-synthetic benchmarks, additional real external guardrails and a live-agent

run, deployment key-management analysis, and independent replication on further agent stacks.

11 CONCLUSION

Input guardrails will sometimes miss. The VCA prototype evaluates and attests what happened downstream after the miss: whether untrusted context gained authority, whether unauthorized tools executed, whether unsafe output was exported, and whether the evidence verifies offline. The contribution is not jailbreak immunity. It is a bounded systems pattern for converting post-guardrail containment from a verbal assurance into a machine-checkable artifact.

REFERENCES

- [1] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating Prompt Injections by Design. arXiv preprint arXiv:2503.18813. <https://arxiv.org/abs/2503.18813>
- [2] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. NeurIPS 2024 Datasets and Benchmarks Track. <https://openreview.net/forum?id=m1YYAQjO3w>

- [3] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISeC '23)*. Association for Computing Machinery, New York, NY, USA, 79–90. <https://doi.org/10.1145/3605764.3623985>
- [4] Simon Josefsson and Ilari Liusvaara. 2017. Edwards–Curve Digital Signature Algorithm (EdDSA). RFC 8032. <https://www.rfc-editor.org/rfc/rfc8032>
- [5] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. 2024. Formalizing and Benchmarking Prompt Injection Attacks and Defenses. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, USA, 1831–1847. <https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei>
- [6] OWASP Gen AI Security Project. 2025. OWASP Top 10 for Large Language Model Applications 2025. <https://genai.owasp.org/llm-top-10/>. Accessed 2026-06-24.
- [7] Anders Rundgren, Bret Jordan, and Samuel Erdtman. 2020. JSON Canonicalization Scheme (JCS). RFC 8785. <https://www.rfc-editor.org/rfc/rfc8785>

GENERATIVE AI USAGE

Generative AI tools were used as writing and engineering assistants during development and drafting. The authors remained responsible for all claims, experiments, code review, evidence interpretation, and final manuscript content. No confidential peer-review material was shared with AI tools.